BikeGo

AI-Powered Bike Taxi & Local Mobility Platform

Project Report & GitHub Documentation

Project type	Smart mobility / transport technology
Core idea	Connect passengers with nearby bike-taxi riders using intelligent matching and transparent fares.
Primary users	Passengers, riders, and platform administrators
Project status	Concept and proposed system design

Prepared as a starter project document. Replace proposed features, technology choices, and implementation status with verified details as the software develops.

Contents

1. Executive Summary

2. Problem Statement

3. Goals and Scope

4. Proposed Features

5. User Roles and User Journey

6. AI and Intelligent Mobility

7. System Architecture

8. Suggested Technology Stack

9. Data Model

10. Safety, Privacy and Security

11. API Design (Proposal)

12. Development Roadmap

13. Testing and Success Metrics

14. Risks and Mitigations

15. Future Enhancements

16. GitHub Repository Structure

17. Conclusion

Appendix: README Starter

1. Executive Summary

BikeGo is a proposed AI-powered bike-taxi and local mobility platform intended to make short-distance urban travel faster, affordable, and convenient. The platform connects passengers who need a ride with available nearby riders, provides estimated fares and arrival times, and supports trip tracking and digital trip records.

The system is designed around three interfaces: a passenger application, a rider application, and an administrative dashboard. Intelligent features may help rank nearby riders, estimate trip duration, forecast demand, and detect unusual trip patterns. These capabilities are design goals, not claims that a working AI model has already been trained or deployed.

Important: this document describes a concept and proposed architecture. Actual features, integrations, compliance, and implementation status should be updated to match the repository.

2. Problem Statement

People making short trips may face traffic congestion, uncertain fares, difficulty finding transport quickly, and limited visibility into a trip once it begins. Riders can face uneven demand and wasted time waiting between bookings.

BikeGo proposes a digital workflow that shows an estimated fare before booking, matches a passenger with an available rider, shares trip status, and records completed journeys. The platform should prioritize safety, transparency, accessibility, and responsible handling of location data.

3. Goals and Scope

Objectives

• Reduce friction in booking short-distance rides.

• Show an estimated fare and arrival time before confirmation.

• Help riders discover nearby trip requests and manage availability.

• Provide trip status, receipts, feedback, and support workflows.

• Use data-driven tools to improve matching and demand planning over time.

Initial scope

The minimum viable product (MVP) can include account registration, pickup/drop-off selection, fare estimation, ride requests, rider acceptance, trip status updates, and trip history. Live maps, payment gateways, identity verification, emergency workflows, and AI-based prediction require suitable providers, testing, and operational policies.

Out of scope for the initial prototype

A production-grade dispatch network, guaranteed arrival times, real-time fraud detection, autonomous vehicles, and availability in every city are not assumed.

4. Proposed Features

Passenger application

• Sign up and manage a profile.

• Choose pickup and destination on a map or by address.

• View estimated distance, fare range, and rider arrival estimate.

• Request, cancel where permitted, and follow ride status.

• View trip history, receipts, ratings, and support options.

Rider application

• Manage profile and availability status.

• Receive nearby ride requests with pickup and destination details appropriate to the workflow.

• Accept or decline requests and update trip progress.

• Review completed trips and earnings summaries.

Admin dashboard

• Review user reports and support cases.

• Manage rider onboarding and account status.

• Monitor aggregate trip, cancellation, and service-quality metrics.

• Apply documented access controls and audit sensitive actions.

5. User Roles and User Journey

Roles

Passenger: requests and tracks trips. **Rider:** receives and completes accepted trips. **Administrator/support agent:** handles onboarding, reports, disputes, and platform operations.

Example passenger journey

1) Open BikeGo and set pickup/drop-off. 2) Review estimated fare and ride details. 3) Confirm the request. 4) View matching and rider assignment status. 5) Verify rider and vehicle details before boarding. 6) Follow the trip status. 7) Complete the trip, pay through an enabled method, and submit feedback.

Suggested ride states

REQUESTED → MATCHING → ACCEPTED → RIDER_ARRIVING → IN_PROGRESS → COMPLETED. Alternative states include CANCELLED, EXPIRED, and SUPPORT_REVIEW. State changes should be validated by the backend rather than trusted solely from the mobile client.

6. AI and Intelligent Mobility

AI should support decisions, not make opaque or unsafe decisions without oversight. Start with explainable rules and introduce machine-learning models only when representative, consented, and appropriately governed data is available.

Potential AI use cases

• **Rider matching:** rank eligible riders by estimated pickup time, distance, availability, and service constraints.

• **ETA estimation:** combine map-provider estimates with historical trip durations when data quality is sufficient.

• **Demand forecasting:** estimate busy zones and time windows from aggregated historical bookings.

• **Fare guidance:** estimate trip cost using distance, time, local rules, and clearly disclosed pricing logic.

• **Anomaly flags:** identify unusual patterns for human review; avoid automatically penalizing users based only on a model score.

Example matching score (illustrative)

```
score = 0.50 × normalized_pickup_time\n + 0.25 × normalized_distance\n + 0.15 ×
rider_reliability\n + 0.10 × vehicle_eligibility
```

This formula is a placeholder for a prototype, not a validated production model. Weights must be tested for fairness, safety, and local operating conditions. Do not use protected traits or sensitive personal data for ranking.

7. System Architecture

A practical architecture separates client interfaces, backend business rules, data storage, and external providers. The backend should remain the authority for booking state, fare rules, permissions, and trip history.

Logical components

• **Passenger app / web client:** booking, fare preview, trip status, history.

• **Rider app:** availability, request handling, navigation handoff, trip updates.

• **API backend:** authentication, ride lifecycle, matching, fare calculation, notifications.

• **Database:** users, rider profiles, rides, payments references, ratings, support cases.

• **External services:** maps/geocoding, push notifications, payment gateway, optional SMS/email.

• **Analytics/AI service:** aggregate metrics and separately evaluated prediction services.

8. Suggested Technology Stack

Choose tools based on team experience and deployment needs. This is one possible stack, not a requirement.

• **Mobile:** Flutter or React Native for cross-platform development.

• **Web/admin:** React with TypeScript.

• **Backend:** Node.js with Express/NestJS, or Python with FastAPI.

• **Database:** PostgreSQL; consider PostGIS for geographic queries.

• **Maps:** a licensed maps/geocoding/directions provider.

• **Authentication:** managed identity provider or secure server-side authentication.

• **Deployment:** a cloud platform with secret management, logging, monitoring, and backups.

Never commit API keys, signing credentials, database passwords, real user location histories, or payment secrets to GitHub. Use environment variables and a committed .env.example containing placeholders only.

9. Data Model

The following entities are a starting point. Use generated IDs, timestamps, validation, and appropriate access rules.

Core entities

• **User:** id, display_name, contact_reference, role, account_status, created_at.

• **RiderProfile:** user_id, verification_status, vehicle_type, vehicle_registration_reference, availability_status.

• **Ride:** id, passenger_id, rider_id, pickup coordinates/reference, destination coordinates/reference, estimated_fare, final_fare, status, created_at, completed_at.

• **RideEvent:** id, ride_id, event_type, timestamp, actor_id.

• **PaymentRecord:** id, ride_id, provider_reference, amount, currency, status; do not store raw card data.

• **Rating:** id, ride_id, reviewer_id, score, optional_comment, created_at.

Minimize collection of precise location data, restrict who can access it, define retention periods, and avoid putting live user records in demo datasets.

10. Safety, Privacy and Security

• Use encrypted transport (HTTPS/TLS) and secure credential storage.

• Apply role-based authorization on every sensitive backend endpoint.

• Collect only data needed for the ride and service operation.

• Explain location permissions and request them only when needed.

• Use verified rider/vehicle workflows appropriate to the operating jurisdiction.

• Provide a clear emergency/support process and escalation ownership before launch.

• Keep audit records for administrative actions and protect them from tampering.

• Rate-limit authentication and booking endpoints; validate all user input.

• Use payment providers; do not store raw card details.

• Test for common web/mobile security issues and have an incident response plan.

A real bike-taxi service must assess applicable transport, licensing, insurance, consumer-protection, tax, accessibility, and data-protection requirements for each location. This report is not legal advice and does not claim that BikeGo is approved to operate.

11. API Design (Proposal)

Example REST endpoints for a future backend. These are illustrative contracts and do not imply that endpoints currently exist.

```
POST /api/v1/auth/register\nPOST /api/v1/auth/login\nGET /api/v1/users/me\nPOST
/api/v1/rides/estimate\nPOST /api/v1/rides\nGET /api/v1/rides/{rideId}\nPOST
/api/v1/rides/{rideId}/accept\nPOST /api/v1/rides/{rideId}/events\nPOST
/api/v1/rides/{rideId}/cancel\nGET /api/v1/users/me/rides
```

Require authentication where appropriate, validate ownership and role, use idempotency for booking/payment operations, and return consistent error codes. Do not expose another user's private trip information.

12. Development Roadmap

Phase 1 — Prototype

• Create wireframes, user stories, and a basic responsive interface.

• Implement sample ride booking and state transitions with mock data.

• Write setup instructions, tests, and API documentation.

Phase 2 — MVP

• Add real authentication, persistent database, role-based access, and validated fare estimates.

• Integrate maps and notifications after provider selection and configuration.

• Add rider onboarding, support workflows, and trip receipts.

Phase 3 — Pilot

• Run a limited, supervised pilot after legal, safety, insurance, and operational checks.

• Measure matching performance, trip completion, cancellations, and user feedback.

Phase 4 — Intelligence and scale

• Evaluate demand forecasting and ETA models using quality-controlled data.

• Monitor model drift, fairness, security, costs, and reliability.

13. Testing and Success Metrics

Testing checklist

• Unit tests for fare calculation, permissions, and ride-state transitions.

• Integration tests for booking, rider acceptance, cancellation, and trip completion.

• Authorization tests proving passengers cannot access unrelated rides.

• Failure tests for network loss, duplicate requests, payment errors, and unavailable riders.

• Accessibility, usability, performance, and security testing before deployment.

Possible metrics

Median time to rider assignment; pickup ETA error; booking completion rate; cancellation rate; ride completion rate; crash/error rate; support resolution time; passenger and rider satisfaction; and safety incidents. Define measurement windows and baselines before interpreting results.

14. Risks and Mitigations

• **Incorrect ETA or fare:** show estimates clearly, explain fare rules, and monitor error ranges.

• **Insufficient rider supply:** display availability honestly and avoid promising guaranteed matches.

• **Location-data exposure:** minimize retention, enforce access controls, and audit access.

• **Unsafe or fraudulent activity:** verification, reporting, human review, and clear escalation procedures.

• **Biased AI recommendations:** test across relevant areas and groups; provide appeal and human review.

• **Provider outage:** define graceful failure states and fallback procedures.

• **Regulatory uncertainty:** complete location-specific review before any real-world launch.

15. Future Enhancements

• Multilingual UI and accessible design.

• Scheduled rides and multi-stop trip support.

• Demand heatmaps based on aggregated, privacy-preserving data.

• Transparent rider incentives and peak-time planning.

• Digital receipts, coupons, and business travel accounts.

• Public transport connections and other local mobility options.

• Model monitoring dashboard and human review queue.

16. GitHub Repository Structure

A clean repository can separate application code, documentation, tests, and deployment configuration. Adapt this structure to the framework actually used.

```
BikeGo/\n■■■ README.md\n■■■ LICENSE\n■■■ .gitignore\n■■■ .env.example\n■■■ docs/\n■ ■■■
BikeGo_Project_Report.pdf\n■ ■■■ architecture.md\n■ ■■■ api.md\n■■■ apps/\n■ ■■■
passenger-app/\n■ ■■■ rider-app/\n■ ■■■ admin-dashboard/\n■■■ services/\n■ ■■■ api/\n■
■■■ matching/\n■■■ tests/\n■■■ assets/
```

Do not create empty folders just to match this diagram. Keep only the folders that correspond to real work, and update the README with actual setup commands and implemented features.

17. Conclusion

BikeGo outlines a practical direction for a bike-taxi and local mobility platform: convenient booking, transparent estimates, rider matching, trip visibility, and a foundation for data-informed improvements. The safest path is to build a small, testable prototype first, document what works, and add production integrations only after reliability, privacy, safety, and local operating requirements have been addressed.

This report is intended as project documentation for a GitHub repository. It should be revised as the implementation evolves so that planned features are clearly distinguished from completed functionality.

Appendix: README Starter

Copy and adapt the following Markdown into your repository's README.md. Replace placeholders and remove claims that are not true for your codebase.

```
# BikeGo — AI-Powered Bike Taxi & Local Mobility Platform\n\nBikeGo is a proposed mobility
platform concept for connecting passengers with nearby bike-taxi riders. The project explores
ride booking, fare estimates, rider matching, trip status, and responsible use of AI for local
mobility.\n\n## Project status\n\nConcept / prototype in development. Features described in the
documentation may be planned rather than implemented.\n\n## Planned capabilities\n\n- Passenger
ride requests and trip history\n- Rider availability and request handling\n- Fare and
arrival-time estimates\n- Ride lifecycle and support workflows\n- Potential AI-assisted
```

```
matching and demand forecasting\n\n## Technology\n\nUpdate this section with the frameworks, services, and database actually used.\n\n## Getting started\n\nAdd the real installation and run commands for this repository. Do not publish secrets.\n\n## Documentation\n\nSee `docs/BikeGo_Project_Report.pdf` for the project concept, proposed architecture, safety considerations, and roadmap.\n\n## Security and privacy\n\nNever commit API keys, credentials, payment details, or real users' location data. Report security issues privately to the repository maintainer.\n\n## Disclaimer\n\nThis repository is a software project concept/prototype and does not imply that a commercial transport service is licensed or operational.
```